



[Home](#)

[Service](#)

[About Us](#)

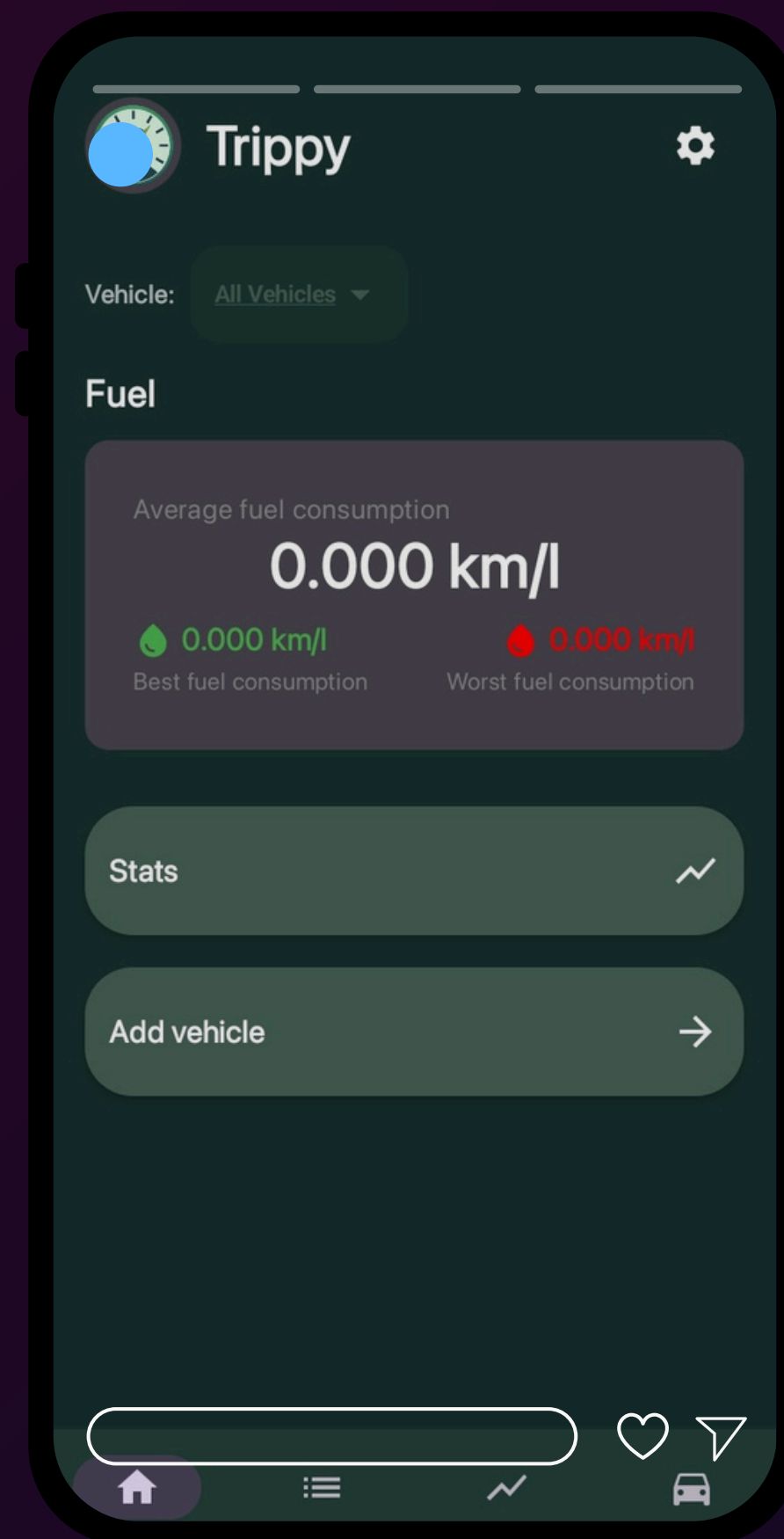
[Contact](#)



Mobile App

DEVELOPMENT





TRIPPY :

The Trippy is an easy-to-use smartphone software made to assist vehicle owners in effectively tracking and controlling their fuel consumption. Budgeting and maintaining the performance of your vehicle depend on you monitoring your fuel usage and associated costs, whether you drive a car, motorcycle, or truck. With the use of graphic data, this app offers a practical way to analyze spending over time, calculate miles, register gasoline fill-ups, and learn more about fuel efficiency.

App Purpose :

To help users:

- Track fuel fill-ups.
- Monitor mileage and fuel efficiency.
- View expenses over time.
- Set reminders for maintenance based on mileage

KEY FEATURES

01

Add Fuel Entry: Date, fuel amount (liters/gallons), cost, and odometer reading.

02

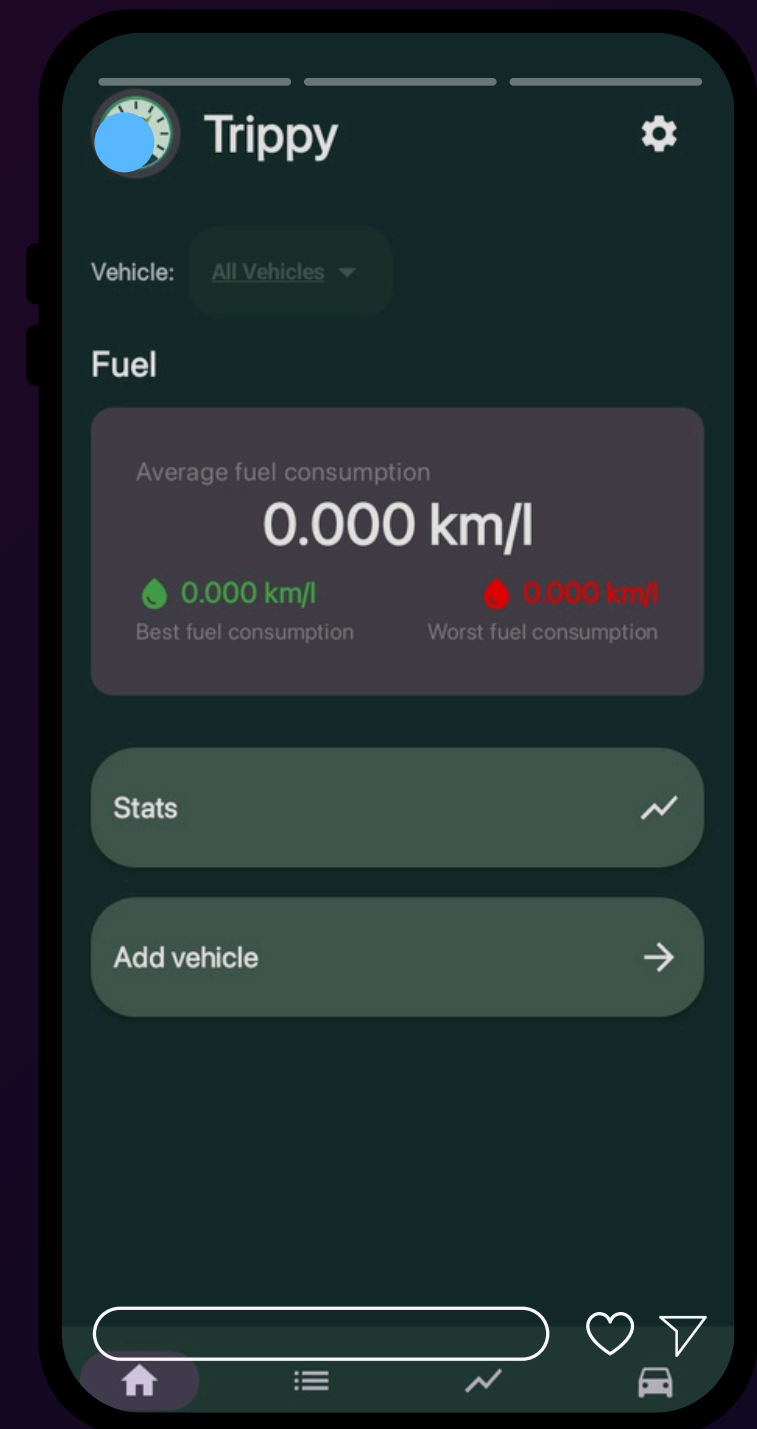
View History: List of past fuel entries.

03

Statistics Dashboard : Charts of cost, distance, and efficiency over time.

04

Fuel Efficiency Calculation: Based on odometer differences and fuel used.





Home

Service

About Us

Contact



Activity 01

Trippy

Vehicle: All Vehicles

Fuel

Average fuel consumption

0.000 km/l

0.000 km/l

0.000 km/l

Best fuel consumption

Worst fuel consumption

Stats

Add vehicle

Activity 02

Settings

0 km/l

umption

Activity 03

Mileage log

Filter by vehicle: All Vehicles

No mileage logs yet.

Activity 04

Cancel

Mileage log

Save

VEHICLE

Choose vehicle

MAIN

Trip meter

Fuel (l)

Fuel type

Fuel price / Litres

Total cost

Date

5/22/25, 6:19 PM

☐ Full tank

Activity 05

Mileage log

Filter by vehicle: All Vehicles

May 2025

5/22/25, 6:22 PM

₱90.00

2.000 l → ₱45.00/l



Activity 06

Activity 08

Cancel

Setup vehicle

Save

Name

Description

UNITS

Distance

Kilometres

Fuel unit

Litres

Consumption

l/100km

FUEL TYPE

Fuel type

Not set

Bi-fuel vehicle (2 tanks)

TANK CAPACITY (OPTIONAL)

Tank capacity

OPTIONAL

Make

Activity 07

Profile picture

Edit vehicle

Save

Name

Rusi Surf 110

Description

Semi Automatic Motorcycle

UNITS

Distance

Fuel unit

Consumption

Kilometres

Litres

l/100km

FUEL TYPE

Fuel type

Not set

Bi-fuel vehicle (2 tanks)

☐

TANK CAPACITY (OPTIONAL)

Tank capacity

2.5 Liters

OPTIONAL

Make

Model

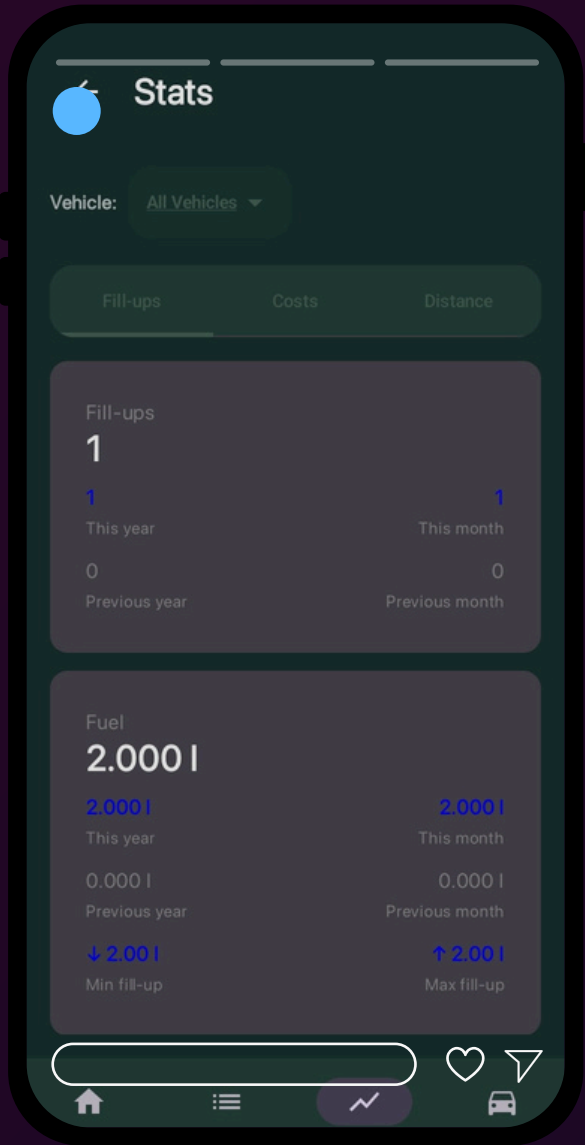
☐

☐

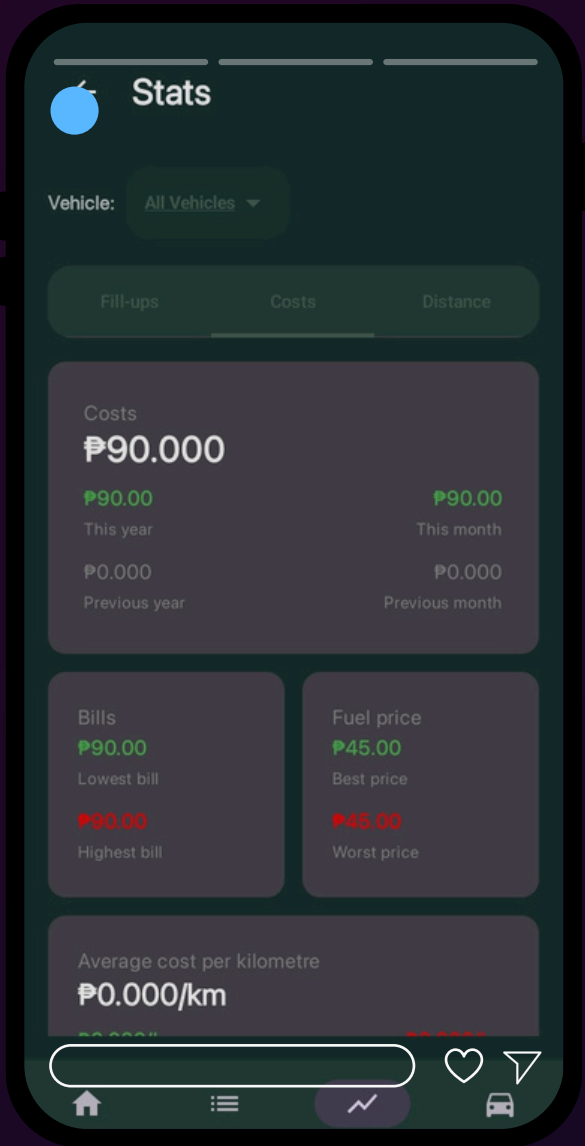
Activity 09

Activity 10

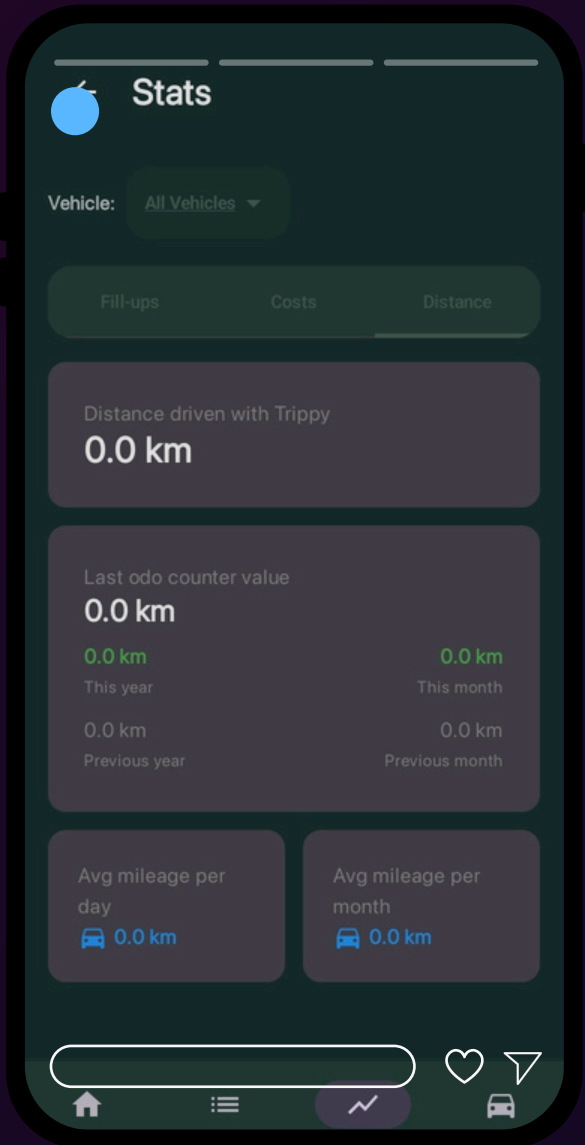
The screenshot displays the 'Trippy' app interface. At the top, there's a header with the app name 'Trippy' and a settings gear icon. Below the header, a 'Vehicle:' label is followed by a dropdown menu currently showing 'All Vehicles'. The main section is titled 'Fuel' and features a large display showing '0.000 km/l'. Below this, two smaller metrics are shown: 'Best fuel consumption' with a green drop icon and 'Worst fuel consumption' with a red drop icon, both displaying '0.000 km/l'. A 'Stats' button with a line graph icon is located below the fuel summary. At the bottom, there's an 'Add vehicle' button with a right-pointing arrow. The bottom navigation bar includes a home icon, a list icon, a line graph icon, a heart icon, and a car icon.



Activity 11



Activity 12



Activity 13

[Home](#)[Service](#)[About Us](#)[Contact](#)

Technologies/Frameworks Used:

01

LANGUAGE : JAVA & KOTLIN

02

DATABASE : ROOM (FOR OFFLINE
LOCAL STORAGE)

03

UI DESIGN : XML LAYOUTS

04

CHARTS : MPANDROIDCHART

05

STORAGE : SHAREDREFERENCES (FOR
SETTINGS), ROOM DB (FOR FUEL LOGS)



LESSON LEARNED:

The experience of creating a Fuel Tracker App in Android Studio was satisfying and hard, and it provided many useful insights into the field of mobile app development. We developed a stronger comprehension of technical ideas, user-centered design, and the software development process as a whole during the project.

Effectively planning and structuring the app's data structure was one of the most important skills we took away. We had to carefully assess what data the app would retain, including gasoline quantity, odometer readings, cost, and date of entry, before implementing any code. We were able to comprehend the importance of clear and rational data modeling through the design of the Room database schema. It provided the framework for precise fuel efficiency estimates and guaranteed seamless data processing across the application.

At last, the process of testing, debugging, and refining the application gave us greater confidence in our ability to use development tools like the Android Emulator and Logcat. It also underlined how crucial time and scope management are. We discovered that the most important functionality should come first, followed by frills like data visualization and maintenance alerts.

